

# METODI PER IL RITROVAMENTO DELLE IN- FORMAZIONI

*Alessandro Carella*

## INDICE

|                                                           |    |
|-----------------------------------------------------------|----|
| 1. Introduzione .....                                     | 3  |
| 1.1. Gestire la conoscenza .....                          | 3  |
| 1.2. Acquisizione della conoscenza .....                  | 3  |
| 1.2.1. Data Retrieval .....                               | 4  |
| 1.2.2. Information Retrieval .....                        | 5  |
| 1.2.3. Data mining .....                                  | 5  |
| 1.2.4. Il processo KDD .....                              | 5  |
| 1.2.5. Text Mining .....                                  | 5  |
| 1.3. Ritrovamento delle informazioni intelligenti .....   | 7  |
| 1.3.1. Ritrovamento delle informazioni .....              | 7  |
| 1.3.2. Come le persone cercano informazioni .....         | 7  |
| 1.4. Sistemi di ritrovamento delle informazioni .....     | 9  |
| 1.4.1. Rilevanza .....                                    | 9  |
| 1.4.2. Ritrovamento delle informazioni intelligenti ..... | 9  |
| 1.4.3. Natural Language Processing .....                  | 10 |
| 1.4.4. Machine learning .....                             | 11 |
| 2. Boolean and Vector Space Retrieval Models .....        | 11 |
| 2.1. Retrieval Models .....                               | 11 |
| 2.2. Definizione formale dei modelli IR .....             | 11 |
| 2.3. Modello di ritrovamento booleano .....               | 12 |
| 2.3.1. Vettori di incidenza .....                         | 13 |
| 2.3.2. Indice invertito .....                             | 13 |
| 2.3.3. Fasi dell'Indicizzatore .....                      | 14 |
| 2.3.4. Elaborazione della query: Operazione AND .....     | 15 |
| 2.3.5. Query booleane: corrispondenza esatta .....        | 16 |
| 2.3.6. Problemi del modello booleano .....                | 17 |
| 2.4. Fasi di preprocessing .....                          | 17 |
| 2.4.1. Tokenizzazione .....                               | 18 |

## INTRODUZIONE

### 1.1. Gestire la conoscenza

Gestire la conoscenza significa raccogliere la conoscenza, organizzarla, distribuirla e renderla accessibile a chi ne ha bisogno (nel momento e nel posto in cui serve) al fine di:

- risparmiare tempo,
- migliorare la qualità dei servizi,
- ridurre i tempi di accesso all'informazione ed alla fruizione dei servizi.

La conoscenza è un capitale:

- intangibile,
- volatile,
- difficile da concretizzare e conservare.

Circa il 90% dei dati presenti nei database del mondo è in forma non strutturata. Gli obiettivi sono quelli di costruire sistemi in grado di processare documenti in linguaggio naturale e ritrovare conoscenza da basi di dati in forma testuale.

### 1.2. Acquisizione della conoscenza

Il processo di acquisizione della conoscenza dal testo in linguaggio naturale è chiamato **text mining**. La classificazione delle tecniche di estrazione ed esplorazione dei dati si basa sull'incrocio di due dimensioni fondamentali: l'obiettivo dell'utente e la natura dei dati analizzati.

- L'approccio dell'utente:
  - **Search (Goal-oriented)**: l'utente ha un obiettivo e sa cosa sta cercando. Formula una query per recuperare informazioni o documenti specifici.
  - **Discover (Opportunistic)**: l'utente ha un approccio esplorativo. Non cerca un dato esatto noto a priori, ma vuole «scoprire» pattern nascosti, tendenza, anomalie o estrarre nuova conoscenza dai dati.
- Il formato dei dati:
  - **Structured Data**: Dati strutturati, tipicamente organizzati in schemi rigidi.
  - **Unstructured Data**: Dati non strutturati, composti prevalentemente da testo libero in linguaggio naturale, privi di uno schema predefinito.

Incrociando queste dimensioni si ottengono quattro diverse aree di studio e applicazione:

- **Data Retrieval (Dati Strutturati + Search)**: consiste nel recuperare record esatti da un database strutturato. Si basa su logica deterministica: un record corrisponde o non corrisponde alla query,
- **Information Retrieval (Dati non Strutturati + Search)**. È la ricerca di informazioni all'interno di collezioni di documenti testuali. Poiché il testo non è strutturato, il sistema restituisce un ranking di documenti in base al loro grado di «rilevanza» rispetto alla query.
- **Data Mining (Dati strutturati + Discover)**. È l'applicazione di algoritmi statistici e di macchine di learning su grandi database strutturati per scoprire pattern ricorrenti, regole di associazione o per addestrare modelli predittivi.
- **Text Mining (Dati non strutturati + Discover)**. È il processo di derivazione di informazioni di alta qualità, pattern e nuova conoscenza partendo da enormi quantità di testo libero.

Tutto ciò è mostrato nella seguente figura.

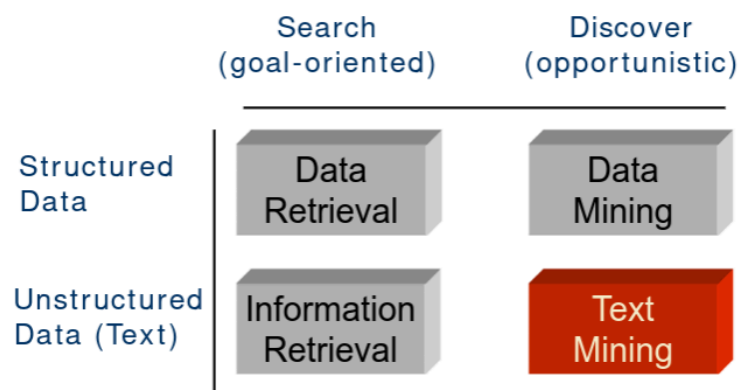


Figura 1: Processo di acquisizione della conoscenza

### 1.2.1. Data Retrieval

Come detto in precedenza, il processo di data retrieval è il ritrovamento di un record in un database strutturato.

|                          |                                                                                        |
|--------------------------|----------------------------------------------------------------------------------------|
| Database Type            | Structured                                                                             |
| Search Mode              | Goal-driven                                                                            |
| Atomic entity            | Data Record                                                                            |
| Example Information Need | "Find a Japanese restaurant in Boston that serves vegetarian food."                    |
| Example Query            | "SELECT * FROM restaurants WHERE city = boston AND type = japanese AND has_veg = true" |

Figura 2: Data Retrieval

### 1.2.2. Information Retrieval

Il processo di information retrieval è la ricerca di un'informazione rilevante in una sorgente di dati non strutturati (tipicamente in formato testo).

|                                 |                                                                     |
|---------------------------------|---------------------------------------------------------------------|
| <b>Database Type</b>            | Unstructured                                                        |
| <b>Search Mode</b>              | Goal-driven                                                         |
| <b>Atomic entity</b>            | Document                                                            |
| <b>Example Information Need</b> | "Find a Japanese restaurant in Boston that serves vegetarian food." |
| <b>Example Query</b>            | "Japanese restaurant Boston" or Boston->Restaurants->Japanese       |

Figura 3: Information Retrieval

### 1.2.3. Data mining

Il processo di data mining permette di scoprire nuova conoscenza attraverso l'analisi di dati.

|                                 |                                                                          |
|---------------------------------|--------------------------------------------------------------------------|
| <b>Database Type</b>            | Structured                                                               |
| <b>Search Mode</b>              | Opportunistic                                                            |
| <b>Atomic entity</b>            | Numbers                                                                  |
| <b>Example Information Need</b> | "Show trend over time in # of visits to Japanese restaurants in Boston " |

Figura 4: Data Mining

### 1.2.4. Il processo KDD

Il processo KDD, acronimo per *Knowledge Discovery from Databases*, è un processo non banale di identificazione valida, nuova e potenzialmente, e ultimamente di comprensibili pattern nei dati.

### 1.2.5. Text Mining

Il processo di text mining è la scoperta di nuova conoscenza attraverso l'analisi del testo.

|                                 |                                                                                    |
|---------------------------------|------------------------------------------------------------------------------------|
| <b>Database Type</b>            | Unstructured                                                                       |
| <b>Search Mode</b>              | Opportunistic                                                                      |
| <b>Atomic entity</b>            | Language feature or concept                                                        |
| <b>Example Information Need</b> | "Find the types of food poisoning most often associated with Japanese restaurants" |
| <b>Example Query</b>            | Rank <b>diseases</b> found associated with "Japanese restaurants"                  |

Figura 5: Text Mining

Il processo di text mining si può dividere nei seguenti passi:

1. **Text Preprocessing (Pre-elaborazione del testo)**. Coinvolge l'analisi *sintattica* e *semantica* per preparare i dati in un formato leggibile dalle fasi successive.
2. **Features Generation (Generazione delle Feature / Trasformazione del testo)**. Consiste nel trasformare il testo pre-elaborato in una rappresentazione matematica o strutturata. L'approccio principale è il **Bag of words** che ignora l'ordine delle parole e la grammatica, concentrandosi sulla presenza e la molteplicità dei termini.
3. **Features Selection (Selezione delle feature)**. La fase precedente genera uno spazio vettoriale di dimensioni enormi, questa fase seleziona solo le caratteristiche più rilevanti e informative. Si basa su tecniche come:
  - **simple counting**: metodi basati sul conteggio semplice delle frequenze dei termini.
  - **statistics**: valutazioni statistiche per capire quali feature hanno il maggiore potere discriminante.
4. **Text/data mining**. È il cuore del processo, dove si applicano gli algoritmi di macchine di machine learning ai dati ormai strutturati e filtrati. Si divide in due paradigmi:
  - **Classification (Supervised learning)**. Apprendimento supervisionato, dove si addestra un modello su testi già etichettati per classificare nuovi documenti.
  - **Clustering (Unsupervised learning)**. Apprendimento non supervisionato, dove l'algoritmo raggruppa autonomamente i documenti in base alle loro similarità intrinseche, senza avere categorie predefinite.
5. **Analyzing result (Interpretazione e valutazione)**. L'ultima fase in cui i pattern estratti vengono analizzati e interpretati per estrarre il vero e proprio «valore» o «conoscenza». Questa fase prevede un **meccanismo di feedback**. L'interpretazione dei risultati può portare a rivalutare e modificare i parametri di qualsiasi delle fasi precedenti per migliorare l'output finale.

Il text mining è importante anche nell'impresa, in quanto il processo di estrazione di conoscenza, precedentemente sconosciuta, da fonti testuali è utilizzabile per prendere decisioni aziendali. Inoltre, permette di organizzare e di categorizzare scoprendo tendenze e apprendendo concetti. Inoltre, il text mining nell'impresa è anche necessario in quanto scoprire tendenze, idee, opinioni e gusti degli utenti sta diventando sempre più impegnativo. Ci sono tante fonti da analizzare come e-mail, forum, articoli, ecc, e l'obiettivo è quello di analizzare migliaia di testi in pochi secondi raggruppandoli in funzione del loro contenuto.

### 1.3. Ritrovamento delle informazioni intelligenti

#### 1.3.1. Ritrovamento delle informazioni

Il ritrovamento delle informazioni, oppure *information Retrieval*, *IR*, si occupa della rappresentazione, dell'archiviazione, organizzazione e accesso a elementi informativi, come documenti, pagine Web, cataloghi online, record strutturati e oggetti multimediali. L'ambito di ricerca iniziale nell'area del ritrovamento delle informazioni riguardava l'indicizzazione del testo e la ricerca di documenti utili in una collezione. Cercare pagine nel web è un compito non banale in quanto bisogna prima ritrovare documenti relativi ad una data query e poi ritrovare, in modo efficiente, informazioni da un grande set di documenti.

Il compito del ritrovamento delle informazioni è quello di trovare una classifica di un insieme di documenti che sono relativi ad una data query. Data la query dell'utente l'obiettivo di un sistema di ritrovamento delle informazioni è quello di ritrovare informazioni utili o rilevanti per l'utente. Il sistema IR deve classificare le informazioni in accordo con un grado di rilevanza alla query data dall'utente.

#### 1.3.2. Come le persone cercano informazioni

L'interazione degli utenti con le interfacce di ricerca è influenzata da diversi fattori. Nel campo dell'information retrieval, si fa, inoltre, una distinzione fondamentale tra le modalità con cui si cerca l'informazione.

Il modo in cui un utente interagisce con un'interfaccia di ricerca varia in funzione di tre elementi:

- **il tipo di compito:** l'obiettivo pratico che l'utente deve portare a termine.
- **l'esperienza nel dominio:** il livello di conoscenza pregressa che chi cerca l'informazione possiede riguardo allo specifico argomento trattato.
- **tempo e sforzo:** la quantità di tempo e di impegno cognitivo che l'utente ha a disposizione o è disposto a investire nel processo.

Come detto prima, si opera una distinzione principale tra due approcci alla ricerca:

- **information lookup**, ricerca mirata;
- **exploratory search**, ricerca esplorativa.

I compiti basati sull'information lookup presentano le seguenti caratteristiche:

- sono assimilabili al recupero di dato di fatto o alla ricerca di risposte dirette a domande specifiche;
- il bisogno dell'utente può essere soddisfatto ottenendo pezzi discreti di informazioni, come numeri, date, siti Web, ecc.;
- questa modalità si sposa molto bene con i paradigmi di interazione standard offerti dai classici motori di ricerca Web.

L'exploratory search, invece, è divisa in compiti di apprendimento e compiti investigativi. La ricerca di apprendimento richiede più di singole coppie query-risposta e richiede a chi effettua la ricerca di spendere tempo per: analizzare e leggere molteplici elementi di informazione e per sintetizzare il contenuto in una nuova forma comprensibile.

Il compito investigativo riguarda un processo a lungo termine che:

- coinvolge interazioni che prendono luogo in un periodo di tempo prolungato;
- può restituire risultati che necessitano di essere valutati criticamente prima di essere integrati nel personale bagaglio professionale e di conoscenza;
- può occuparsi di trovare una grande proporzione di informazioni rilevanti disponibili.

La ricerca di informazioni può essere vista come parte di un processo più grande al quale ci si riferisce come *sensemaking*. Il processo di **sensemaking** è un processo iterativo che consiste nel formulare una rappresentazione concettuale a partire da una vasta collezione di dati. All'interno di questo processo, la maggior parte dello sforzo cognitivo e del tempo dell'utente viene investita nella sintesi, ovvero nella creazione di una «buona rappresentazione» strutturata dell'argomento.

Le attività di sensemaking possono svilupparsi secondo diverse modalità operative:

- **interwoven search, ricerca intrecciata.** In alcuni casi, l'attività di ricerca si intreccia continuamente con l'analisi lungo tutto l'arco del processo.
- **batch process, ricerca a blocchi.** In altri casi, il processo è diviso in fasi distinte: l'utente esegue prima un blocco intensivo di sole ricerche, che viene poi seguito da un blocco separato dedicato interamente all'analisi e alla sintesi dei risultati raccolti.

Degli esempi di compiti di analisi profonda che richiedono l'utilizzo del processo di sensemaking sono:

- **Legal discovery:** il processo in ambito legale di ricerca, raccolta e analisi di grandi quantità di documenti o prove elettroniche.
- **Epidemiologia:** il tracciamento e lo studio della diffusione delle malattie.
- **Analisi del customer care:** lo studio sistematico dei reclami dei clienti con l'obiettivo di migliorare il servizio offerto.

- **Business intelligence:** l'acquisizione e l'analisi di dati complessi per ricavare informazioni strategiche e supportare le decisioni aziendali.

## 1.4. Sistemi di ritrovamento delle informazioni

### 1.4.1. Rilevanza

La rilevanza è un giudizio soggettivo e può includere:

- essere sul giusto argomento;
- avere informazioni recenti;
- soddisfare gli obiettivi dell'utente e del suo uso previsto delle informazioni.

La definizione più semplice di rilevanza è che la query appare parola per parola nel documento. Una nozione più generale, invece, è che le parole nella stringa appaiono frequentemente nel documento, in qualsiasi ordine. Per questo si usa il termine di *bag of words*, intesa come una sacca di parole del quale non importa l'ordine.

Il problema di queste parole chiave è che è possibile non ritrovare documenti rilevanti che includono, però, dei sinonimi di tali parole. Oppure, è possibile ritrovare documenti irrilevanti che possono contenere termini polisemici, ovvero stessa parola ma con significato diverso. Ad esempio, la parola «bat» in inglese può significare sia pipistrello che la mazza da baseball.

### 1.4.2. Ritrovamento delle informazioni intelligente

Un sistema di *intelligent IR* si distingue dai sistemi di base per le seguenti caratteristiche:

- prende in considerazione il *significato* delle parole utilizzate;
- prende in considerazione l'*ordine* delle parole all'interno della query;
- si adatta all'utente basandosi su un feedback che può essere diretto o indiretto;
- nel processo di *relevance feedback*, il sistema raccoglie il feedback fornito, genera una nuova query e ripete l'operazione di recupero delle informazioni.

Analizziamo l'architettura di un sistema IR:

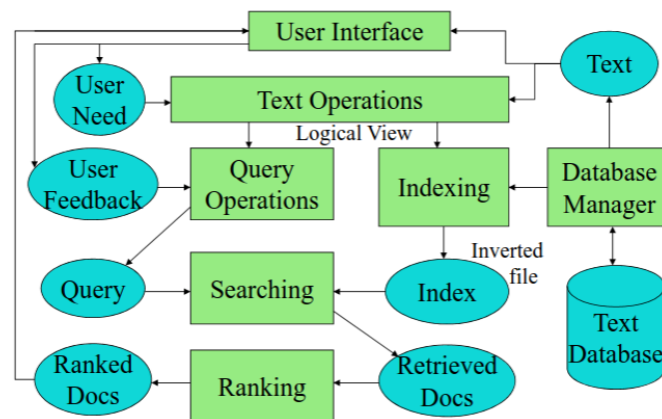


Figura 6: Architettura di un sistema IR

Dalla foto precedente descriviamo le seguenti componenti:

- **Text operations**, le operazioni sul testo. Questa fase si occupa di formare le parole indice, ovvero i *token*. Include operazioni come:
  - la rimozione delle stopwords
  - lo stemming, ovvero ridurre le parole alle loro radici tramite la rimozione di prefissi e suffissi.
- **Indexing**, l'indicizzazione. Questa fase consiste nel costruire un *indice invertito* detto *inverted index* che mappa le parole verso i puntatori dei documenti in cui esse compaiono.
- **Searching**, la ricerca. Questa fase ha lo scopo di recuperare, a partire dall'inverted index, i documenti che contengono uno specifico token di ricerca.
- **Ranking**. Questa fase assegna un punteggio a tutti i documenti recuperati basandosi su una matrice di rilevanza. Questa componente può occuparsi anche del raggruppamento, ovvero trovare elementi in comune per presentare all'utente dei gruppi di documenti.

Le aree di ricerca dei sistemi di intelligent IR riguardano il processing del linguaggio naturale e il machine learning.

#### 1.4.3. Natural Language Processing

Il Natural Language Processing, NLP, nell'ambito dell'Information Retrieval si caratterizza per i seguenti aspetti:

- è focalizzato sull'analisi sintattica, semantica e pragmatica del testo e del discorso in linguaggio naturale.
- la capacità di analizzare la sintassi e la semantica potrebbe consentire un recupero delle informazioni basato sul significato, piuttosto che sulle semplici parole chiave.

L'applicazione del Natural Language Processing al campo dell'Information Retrieval si sta sviluppando lungo diverse direzioni di ricerca principali:

- **Word Sense Disambiguation.** Sviluppo di metodi per determinare il significato corretto di una parola ambigua basandosi sul contesto in cui si trova.
- **Information Extraction.** Utilizzo di metodi volti a identificare specifiche porzioni di informazione all'interno di un documento.
- **Question answering.** Creazione di metodi per fornire risposte a specifiche domande poste in linguaggio naturale ricavandole da ampi corpi di documenti.

#### 1.4.4. Machine learning

Il campo del Machine Learning si concentra sui seguenti aspetti:

- È focalizzato sullo sviluppo di sistemi computazionali che sono in grado di migliorare le proprie prestazioni attraverso l'esperienza.
- **Apprendimento supervisionato (Supervised learning).** Prevede la classificazione automatica di esempi basandosi sull'apprendimento di concetti derivati da esempi di addestramento precedentemente etichettati.
- **Apprendimento non supervisionato (Unsupervised learning).** Consiste nell'utilizzo di metodi automatici per raggruppare esempi non etichettati all'interno di gruppo che risultino significativi.

## BOOLEAN AND VECTOR SPACE RETRIEVAL MODELS

### 2.1. Retrieval Models

Un modello di ritrovamento, detto **retrieval model**, specifica i dettagli di:

- rappresentazione dei documenti;
- rappresentazione delle query
- funzione di ritrovamento, detta **retrieval function**.

Per implementare la funzione di ritrovamento bisogna determinare la nozione di rilevanza che può essere binaria o continua.

Inoltre, determina anche la funzione di classificazione, detta **ranking function** che assegna i punteggi ai documenti relativi ad una data query.

### 2.2. Definizione formale dei modelli IR

Definiamo formalmente i modelli di ritrovamento, *IR models*, come segue:

**Definizione 1.** (Modello IR) Un modello di ritrovamento delle informazioni è una quadrupla  $[D, Q, F, R(q_i, d_j)]$  ove:

1.  $D$  è un insieme di viste logiche per i documenti nella collezione.
2.  $Q$  è un insieme di viste logiche per le informazioni che servono all'utente. Queste viste sono chiamate *query*.
3.  $F$  è un framework per la modellazione dei documenti e query.
4.  $R(q_i, d_j)$  è una funzione di ranking, che confronta la query con il documento e rileva quanto il documento è rilevante rispetto alla query data.

Esistono diversi modelli di ritrovamento, come mostrari nella figura di seguito:

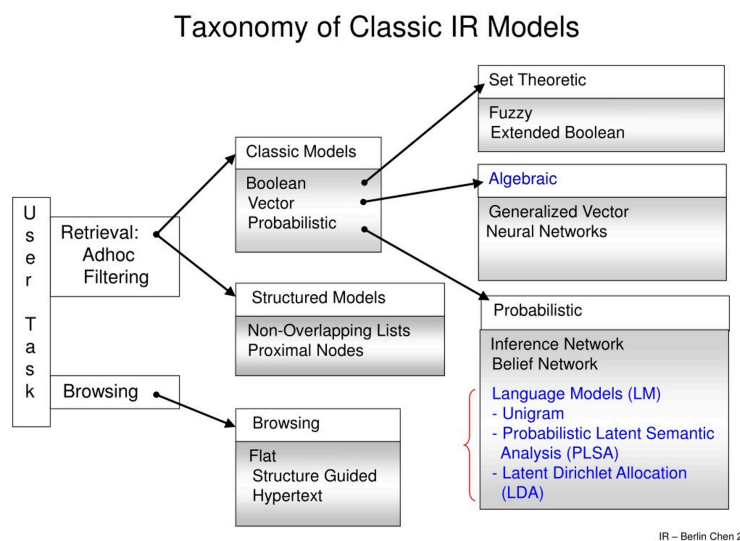


Figura 7: Tassonomia dei modelli di ritrovamento

Cercare foto simile a quella nelle slide

## 2.3. Modello di ritrovamento booleano

Il modello di ritrovamento booleano è basato sulla teoria degli insiemi. Per approcciarsi a questi modelli ci sono dei passi comuni nella fase di preprocessing:

- rimuovere caratteri non voluti, come tag HTML, punteggiature, numeri, ecc.
- dividere il documento in *token* dividendolo sulla base degli spazi;
- ricondurre i token alla radice, detta operazione di *stemming*
- rimuovere comuni parole chiuse, come congiunzioni, articoli, ecc.
- Individuare frasi comuni, possibilmente usando un dizionario specifico di dominio
- costruire gli indici invertiti, detti *inverted index*, che permettono di accedere alla lista dei documenti che contengono una data parola, detta *keyword*.

Nel modello di ritrovamento booleano un documento è rappresentato mediante un **insieme** di parole e le query sono espressioni booleane di keyword collegate

tramite le operazioni di AND, OR e NOT, includendo l'uso di parentesi graffe per indicare lo scopo. In output si ottiene se il documento è rilevante o meno, non ci sono corrispondenze parziali o una classificazione di rilevanza. Quindi la nozione di rilevanza in questo modello è di tipo booleana.

### 2.3.1. Vettori di incidenza

Consideriamo la seguente tabella:

|           | Antony and Cleopatra | Julius Caesar | The Tempest | Hamlet | Othello | Macbeth |
|-----------|----------------------|---------------|-------------|--------|---------|---------|
| Antony    | 1                    | 1             | 0           | 0      | 0       | 1       |
| Brutus    | 1                    | 1             | 0           | 1      | 0       | 0       |
| Caesar    | 1                    | 1             | 0           | 1      | 1       | 1       |
| Calpurnia | 0                    | 1             | 0           | 0      | 0       | 0       |
| Cleopatra | 1                    | 0             | 0           | 0      | 0       | 0       |
| mercy     | 1                    | 0             | 1           | 1      | 1       | 1       |
| worser    | 1                    | 0             | 1           | 1      | 1       | 0       |

Quindi, per ogni termine abbiamo un vettore composto da 0 o 1. Per rispondere ad una query come:

Brutus, Ceaser and NOT Calpurnia

si prendono i vettori per le rispettive parole:

- *Brutus*: 110100;
- *Caesar*: 110111;
- *Calpurnia* (complementato): 101111.

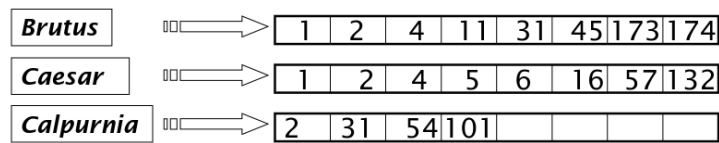
Infine, eseguiamo l'and bit a bit e otteniamo:

$$110100 \text{ AND } 110111 \text{ AND } 101111 = 100100$$

### 2.3.2. Indice invertito

Per ogni termine  $t$ , memorizziamo una lista di tutti i documenti che contengono tale ptermine. Ogni documento è identificato da un **docID** è un numero seriale per i documenti.

Supponiamo di implementare gli indici invertiti utilizzando array di dimensione statica e consideriamo la seguente figura:



Se, ad esempio, si aggiunge la parola *Caesar* al documento 14, non possiamo inserire quest'ultimo nell'array e, quindi, bisogna usare strutture dati in grado di supportare liste di **postings** a dimensione variabile.

In memoria centrale, questo problema si risolve tipicamente utilizzando:

- **Liste concatenate**
- **Array a lunghezza variabile.**

Queste soluzioni dinamiche permettono di aggiungere nuovi **docID** man mano che l'indice cresce, accettando dei compromessi tra l'ottimizzazione dello spazio occupato e la facilità di inserimento. Su disco, invece, è normale e ottimale mantenere una sequenza continua di dati per velocizzare il tempo di lettura.

Inoltre, a prescindere dalla struttura dinamica scelta in memoria, è fondamentale ricordare che i documenti all'interno di ogni lista di **posting** devono sempre essere mantenuti ordinati per **docID** crescente per garantire l'efficienza delle operazioni di intersezione durante le query.

Segue un esempio di costruzione dell'indice invertito:

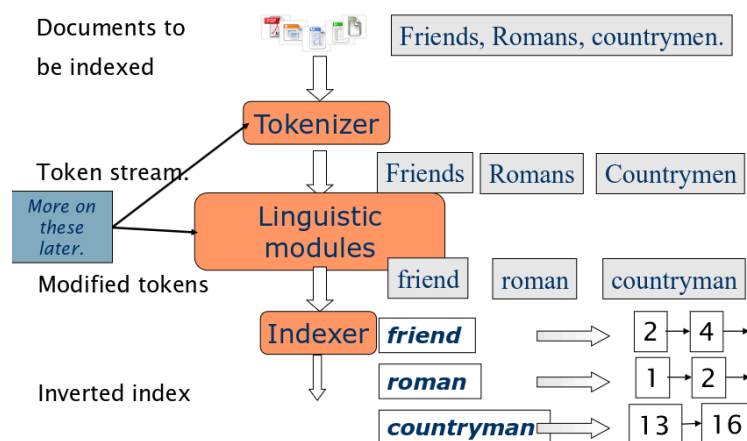


Figura 9: Costruzione dell'indice invertito

### 2.3.3. Fasi dell'Indicizzatore

Analizziamo le passi che compongono l'indicizzatore:

1. Creazione delle coppie (Token, docID)

- Il processo estrae il testo e genera una sequenza formata da coppie di valori: un **token modificato** e l'**ID del documento** in cui quel token compare.
  - I token vengono definiti «modificati» perchè, subiscono una pre-elaborazione prima di essere inseriti nella tabella.
2. **Ordinamento.** Una volta generata la sequenza iniziale di coppie (Token, docID), il sistema procede con l'ordinamento di questi dati. Questo passaggio è identificato come la fase centrale e più importante del processo di indicizzazione. L'algoritmo riorganizza l'intera tabella applicando due criteri di ordinamento in sequenza:
1. **Ordinamento primario per termine:** tutte le coppie vengono ordinate alfabeticamente in base alla colonna del token. In questo modo, tutte le occorrenze della stessa parola vengono raggruppate vicine.
  2. **Ordinamento secondario per docID:** a parità di termine, le tuple vengono ordinate in modo crescente in base all'identificativo del documento.
3. **Dizionario e postings.** Dopo aver ordinato alfabeticamente i termini e i relativi docID, il sistema trasforma la lista lineare nella struttura finale dell'indice invertito attraverso tre operazioni fondamentali:
- **Unione dei termini (Merging):** le entrate multiple dello stesso termine relative a un singolo documento vengono unite. Se una parola compare più volte, non si creano puntatori duplicati allo stesso documento, ma le occorrenze vengono compattate.
  - **Divisione in due strutture (Split):** I dati elaborati vengono formalmente suddivisi nelle due componenti principali: il **Dizionario** e le relative liste di **Postings**.
  - **Aggiunta della Document Frequency:** Per ogni termine nel dizionario viene aggiunta l'informazione sulla *Document Frequency*. Questo valore indica il numero totale di documenti distinti all'interno della collezione in cui quel termine è presente.

#### 2.3.4. Elaborazione della query: Operazione AND

Per comprendere come il sistema estrae i documenti a partire da un indice invertito, consideriamo l'elaborazione di una query con operatore logico AND: **Brutus AND Caesar**.

Le fasi dell'elaborazione sono le seguenti.

1. Localizza il termine **Brutus** all'interno del Dizionario e ne recupera la rispettiva lista di *postings*.
2. Localizza il termine **Caesar** nel Dizionario e ne recupera la relativa lista di *postings*.
3. Esegue un Merge delle due liste di postings per individuare i documenti che contengono contemporaneamente entrambe le parole.

Affinchè l'intersezione sia computazionalmente efficiente, sussistono due vincoli fondamentali:

- I postings all'interno di ciascuna lista devono essere rigorosamente ordinati per docID;
- Le liste devono essere processate in **ordine di lunghezza crescente**, iniziando dalle parole più rare per ridurre da subito il numero di documenti da confrontare.

L'algoritmo di intersezione è il seguente.

#### Algoritmo 1: Intersezione

```
1 Intersect( $p_1, p_2$ )
2    $answer \leftarrow \langle \rangle$ 
3   while  $p_1 \neq \text{NIL}$  and  $p_2 \neq \text{NIL}$ 
4     do if  $\text{docID}(p_1) = \text{docID}(p_2)$ 
5       then  $\text{ADD}(answer, \text{docID}(p_1))$ 
6        $p_1 \leftarrow \text{next}(p_1)$ 
7        $p_2 \leftarrow \text{next}(p_2)$ 
8     else if  $\text{docID}(p_1) < \text{docID}(p_2)$ 
9       then  $p_1 \leftarrow \text{next}(p_1)$ 
10    else  $p_2 \leftarrow \text{next}(p_2)$ 
11  return  $answer$ 
```

#### 2.3.5. Query booleane: corrispondenza esatta

Il modello di ritrovamento booleano si basa sulla capacità di sottoporre al sistema delle query formulate come vere e proprie espressioni booleane.

##### 1. Caratteristiche del modello.

- Le query booleane utilizzano gli operatori logici fondamentali.
- In questo modello, ogni documento viene visto e trattato matematicamente come un semplice *insieme di parole*.
- La sua natura è **precisa**: l'esito della ricerca è puramente binario. Un documento soddisfa esattamente la condizione dettata dalla query oppure non la soddisfa affatto.
- Per via di queste caratteristiche, rappresenta forse il modello più semplice su cui sia possibile costruire un sistema di Information Retrieval.

##### 2. Rilevanza storica e attuale

- È stato lo strumento di retrieval principale in ambito commerciale per circa tre decenni.

- Ancora oggi, molti dei sistemi di ricerca di uso comune si basano su logico booleana. Tra questi figurano la ricerca all'interno delle email, i cataloghi delle biblioteche e la funzione Spotlight di Mac OS X.

### 2.3.6. Problemi del modello booleano

Nonostante la loro semplicità, i modelli di retrieval booleano presentano diverse criticità e limitazioni:

- **Estrema rigidità:** l'utilizzo dell'operatore AND richiede che tutti i termini siano presenti, portando spesso a ottenere troppi pochi risultati. Al contrario, l'operatore OR si accontenta di un qualsiasi termine, rischiando di generare una mole eccessiva di risultati.
- **Difficoltà nell'esprimere richieste complesse.** Risulta complicato esprimere i bisogni informativi complessi degli utenti, poichè queste informazioni devono essere obbligatoriamente tradotte e forzate all'interno di un'espressione booleana.
- **Mancanza di controllo sul volume dei risultati.** È difficile controllare il numero di documenti che il sistema recupera, poichè tutti i documenti che presentano una corrispondenza verranno inevitabilmente restituiti.
- **Difficoltà nel ranking.** Non è possibile generare una classifica di rilevanza accurata, dato che tutti i documenti recuperati soddisfano logicamente la query allo stesso livello.
- **Difficoltà nell'applicazione del relevance feedback.** Risulta problematico implementare meccanismi di feedback. Se l'utente identifica un documento restituito come rilevante o irrilevante, non è chiaro come la query originale debba essere modificata di conseguenza.

## 2.4. Fasi di preprocessing

Quando un sistema si appresta a elaborare un documento grezzo, deve prima determinare alcune sue caratteristiche fondamentali, rispondendo a tre domande:

- **Formato:** in quale formato si trova il file;
- **Lingua:** in quale lingua è scritto il testo;
- **Codifica:** quale codifica dei caratteri viene utilizzata.

Dal punto di vista formale, ognuna di queste sfide rappresenta un vero e proprio **problema di classificazione**. Nonostante la loro natura, nella pratica questi compiti vengono spesso affrontati in modo *euristico*: invece di addestrare modelli complessi, la classificazione viene predetta applicando delle regole semplici. Un esempio di questa implementazione è il riconoscimento della lingua: se nel testo ci sono molte occorrenze della parola «the», allora il documento è in inglese.

Durante il processo di parsing e indicizzazione, i sistemi devono affrontare diverse complicazioni legate alla varietà di formati e lingue:

### 1. la gestione del multilinguismo

- la collezione di documenti da indicizzare può includere testi scritti in molte lingue differenti. Questo implica che un singolo indice potrebbe trovarsi a memorizzare e gestire termini appartenenti a diverse lingue contemporaneamente
2. **documenti e componenti ibridi.** In alcuni casi, un singolo documento o i vari componenti che lo formano possono contenere al loro interno molteplici lingue o formati. Un tipico esempio è un'email scritta in francese che ha come allegato un documento PDF scritto in tedesco.
  3. **Il problema dell'unità documentale.** Alla luce di queste complessità strutturali, sorge una domanda fondamentale: come si definisce esattamente un «documento unitario» da indicizzare? Ci sono diverse interpretazioni a seconda del contesto:
    - è un singolo file?
    - è un'email?
    - è un'email considerata insieme a tutti i suoi allegati?
    - oppure un gruppo di file interconnessi?

#### 2.4.1. Tokenizzazione

La tokenizzazione è il processo di suddivisione del testo grezzo in unità fondamentali e significative. Analizziamone un esempio.

**Esempio 1.** Sia data in input la frase:

Friends, Romans and Countrymen

e in output il sistema elabora il testo e restituisce un insieme di tokens. In questo esempio specifico vengono estratti:

Friends Romans Countrymen

Diamo una definizione formale di token.

**Definizione 2.** (Token) Un **token** è definito come un'istanza di una specifica sequenza di caratteri all'interno del testo.

Ogni token così generato diventa un candidato potenziale per diventare una voce vera e propria all'interno dell'indice. Tuttavia, per diventare una voce definitiva dell'indice, il token deve prima subire un **ulteriore processamento**.

La domanda fondamentale che guida le fasi successive della progettazione di un indicizzatore è: **quali sono i token validi da emettere?**

Nel processo di tokenizzazione sorgono diverse problematiche e ambiguità su come interpretare correttamente i caratteri speciali, i trattini e la punteggiatura. I casi critici principali sono i seguenti:

- **Gestione degli apostrofi e dei genitivi.** Prendendo come esempio la frase «Finland's capital», come deve essere gestito il termine possessivo? Il token risultante deve essere *Finland?*, *Finlands?* oppure *Finland's?*
- **Parole con trattino.** Espressioni come «Hewlett-Packard» devono essere suddivise in due token distinti? Ci sono casi come «state-of-the-art», in cui bisogna decidere se spezzare o meno una sequenza complessa unita da trattini. Esistono parole ambigue come «co-education», o variazioni ortografiche come *lowercase*, *lower-case* e *lower case*. In questi casi, può rilevarsi efficace fare in modo che sia l'utente stesso a inserire eventuali trattini per guidare la ricerca.
- **Nomi composti.** Nomi propri come «San Francisco» devono essere considerati come un unico token oppure come token separati? E soprattutto: con quale criterio logico si decide se un'espressione plurinomiale costituisce un token unico?

### La gestione dei numeri nella tokenizzazione

La gestione delle sequenze numeriche e dei codici alfanumerici rappresenta un aspetto critico durante la tokenizzazione, a causa della grande varietà di formati esistenti.

Ad esempio le date possono essere rappresentate come *3/20/91*, *Mar. 12, 1991* oppure *20/3/91*.

Spesso queste stringhe contengono spazi interni che complicano la separazione dei token. I sistemi di IR meno recenti potrebbero scegliere di non indicizzare affatto i numeri. Tuttavia, indicizzare i numeri è spesso estremamente utile: basti pensare alla ricerca sul web di codici di errore o di *stack trace*. Una possibile soluzione tecnica a questo problema è l'uso degli **n-grammi**. Molto spesso, i metadati associati vengono indicizzati separatamente come **meta-data**.

### Problematiche legate alla lingua

Le scelte di tokenizzazione variano profondamente in base alle specificità linguistiche e morlogiche della lingua in cui è scritto il documento.

#### 1. Caso di lingua francese:

- **Gestione delle elisioni:** un'espressione come *l'ensemble* solleva il dubbio se debba essere considerata come un token unico oppure come due distinti. Le opzioni di suddivisione includono: separare in *L* e *ensemble?*, considerare *L?* oppure espandere in *Le* e *ensamble*.
- **Problemi di corrispondenza (matching).** Spesso si desidera che la ricerca di *l'ensemble* restituisca una corrispondenza anche con forme simili come un *esemble*. Tuttavia, su Google questo comportamento non era garantito fino al 2003, evidenziando le grandi sfide legate all'**internazionalizzazione**.

#### 2. Caso del Tedesco:

- **Composti nominali non segmentati.** In lingua tedesca i sostantivi composti vengono scritti uniti senza spazi. Un esempio sono la parola *Lebensversicherungsgesellschaftangestellter*, che significa letteralmente «impiegato di una compagnia di assicurazioni sulla vita».
  - **L'uso del Compound Splitter.** Per ovviare a questo problema, i sistemi di Information Retrieval in lingua tedesca traggono enormi vantaggi dall'integrazione di un modulo di scomposizione noto come **compound splitter**. L'adozione di questo modulo può incrementare le prestazioni di ricerca per il tedesco fino al 15%.
3. **Caso di cinese e giapponese:**
- Assenza di spazi: il cinese e il giapponese non prevedono spazi tra le parole;
  - Tokenizzazione non unica: a causa di ciò, non è sempre garantita una tokenizzazione univoca;
  - Nel caso del giapponese ci sono *alfabeti multipli mescolati*: la gestione diventa più complessa a causa della presenza contemporanea di più alfabeti intrecciati tra loro. Le date e gli importi si presentano in formati differenti. Inoltre, l'utente finale può scegliere di esprimere la propria query intermettendo utilizzando l'alfabeto hiragana.
4. **Caso di arabo e ebraico**
- *Direzione di scrittura*: l'arabo e l'ebraico si scrivono fondamentalmente da destra a sinistra, sebbene determinati elementi come i numeri siano scritti da sinistra a destra.
  - *Rappresentazione e Unicode*: grazie all'utilizzo di Unicode, la presentazione visiva di superficie risulta complessa a causa della mescolanza di direzioni, ma la forma memorizzata sottostante rimane lineare e diretta.